

COMP9312 — Assignment 1 Solutions

YUHUI LUO (z5565446)

Q1. Disjoint-set forests for connected components

Setup. Union by size: the larger tree's root stays as the root, and the smaller tree's root is attached under it.

Tie-break. When both trees have equal size, the tree whose root is *lexicographically smaller* is treated as larger, so its root becomes the new root.

Path compression. Not applied.

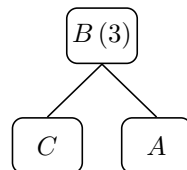
Notation. Each node is a set element; a root is labelled **root(size)** and its children hang below it. Initially eight singletons A, B, C, D, E, F, G, H , each of size 1. Each drawing shows only the non-trivial trees; any vertex not shown remains an isolated singleton.

Edge 1: B–C — both size 1, tie $\Rightarrow$ smaller root B wins.



Components: $\{B, C\}$; singletons A, D, E, F, G, H .

Edge 2: A–C — A (size 1) vs. tree rooted B (size 2); B larger.



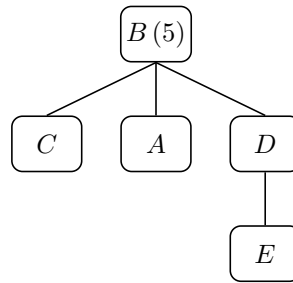
Components: $\{A, B, C\}$; singletons D, E, F, G, H .

Edge 3: D–E — both size 1, tie $\Rightarrow$ smaller root D wins.



Components: $\{A, B, C\}$, $\{D, E\}$; singletons F, G, H .

Edge 4: C–E — tree B (size 3) vs. tree D (size 2); B larger, so D attaches to B .



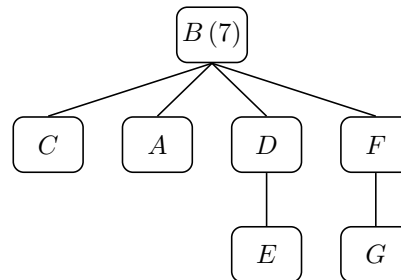
Components: $\{A, B, C, D, E\}$; singletons F, G, H .

Edge 5: F–G — both size 1, tie $\Rightarrow$ smaller root F wins.



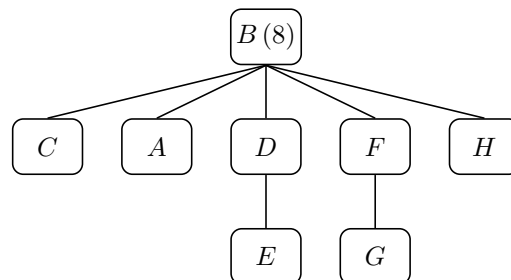
Components: $\{A, B, C, D, E\}$, $\{F, G\}$; singleton H .

Edge 6: E–G — tree B (size 5) vs. tree F (size 2); F attaches to B .



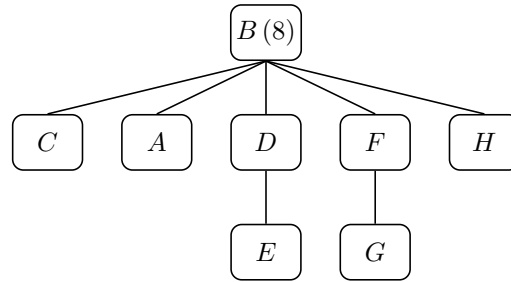
Components: $\{A, B, C, D, E, F, G\}$; singleton H .

Edge 7: H–F — tree B (size 7) vs. H (size 1); H attaches to B .



Components: $\{A, B, C, D, E, F, G, H\}$ — a single component.

Edge 8: G–H — $\text{find}(G) = \text{find}(H) = B$: already in the same set $\Rightarrow$ **no change**; the forest is identical to that after Edge 7.



Components: $\{A, B, C, D, E, F, G, H\}$ — unchanged (still one component).

Answer. All eight elements end up in a single connected component: one tree of size 8 rooted at B , as shown after Edge 7.

Q2. BFS on a grid map

Start $S = (1,1)$. Neighbour scan order **Right** $\rightarrow$ **Down** $\rightarrow$ **Left** $\rightarrow$ **Up**. Visited time = the dequeue order, starting at 0.

(a) Visited times

Dequeue sequence (time : cell):

0:(1,1) 1:(1,2) 2:(1,3) 3:(2,3) 4:(3,3) 5:(3,4) 6:(3,2)
 7:(3,5) 8:(3,1) 9:(3,6) 10:(4,5) 11:(4,1) 12:(5,5) 13:(5,1)
 14:(5,6) 15:(6,5) 16:(5,2) 17:(6,6) 18:(6,4) 19:(5,3) 20:(6,3)

Grid with visited times (# = blocked, inf = passable but unreachable from S):

	c1	c2	c3	c4	c5	c6
r1:	0	1	2	#	inf	inf
r2:	#	#	3	#	#	#
r3:	8	6	4	5	7	9
r4:	11	#	#	#	10	#
r5:	13	16	19	#	12	14
r6:	#	#	20	18	15	17

Cells (1,5) and (1,6) are passable but walled off from S , so BFS never reaches them.

(b) Queries (arbitrary neighbour order)

A FIFO BFS dequeues vertices in non-decreasing **distance from S** (all of level k before any of level $k+1$), regardless of neighbour order. Hence:

- $\text{dist}(x) < \text{dist}(y) \Rightarrow x$ always before $y \Rightarrow$ **true**;
- $\text{dist}(x) > \text{dist}(y) \Rightarrow x$ always after $y \Rightarrow$ **false**;
- $\text{dist}(x) = \text{dist}(y) \Rightarrow$ a neighbour order can put either first $\Rightarrow$ **false**.

Distances from S :

(1,1)=0 (1,2)=1 (1,3)=2 (2,3)=3 (3,3)=4
 (3,2)=5 (3,4)=5

$(3,1)=6$ $(3,5)=6$
 $(4,1)=7$ $(3,6)=7$ $(4,5)=7$
 $(5,1)=8$ $(5,5)=8$
 $(5,2)=9$ $(5,6)=9$ $(6,5)=9$
 $(5,3)=10$ $(6,6)=10$ $(6,4)=10$
 $(6,3)=11$

Query	x	y	$\text{dist}(x)$	$\text{dist}(y)$	Result
1	(1,2)	(3,2)	1	5	true
2	(3,4)	(3,2)	5	5	false (same level)
3	(5,1)	(3,6)	8	7	false (x deeper)
4	(4,5)	(5,6)	7	9	true
5	(6,6)	(5,3)	10	10	false (same level)
6	(3,1)	(5,5)	6	8	true
7	(5,6)	(4,5)	9	7	false (x deeper)
8	(6,4)	(6,3)	10	11	true

Q3. Graph representation and algorithm time complexity

Assume a vertex-indexed **visited/dist** array of size n ($O(1)$ access, $O(n)$ to initialise).

Reasoning per operation:

- **SingleSourceShortestDistances(u)**: unweighted $\Rightarrow$ BFS; cost = $O(n)$ init + enumerating the neighbours of every visited vertex.
- **CountConnectedComponents()**: a full BFS/DFS sweep over all vertices and edges.
- **CountCommonNeighbours(u, v)**: enumerate the neighbours of one vertex and test membership in the other (cost driven by the per-structure membership mechanism).
- **DFS(u)**: same enumeration cost as the BFS sweep from u .

The only structure that cannot enumerate neighbours in $O(\deg(v))$ is the adjacency matrix (a row scan is always $O(n)$), so all of its traversals become $O(n^2)$.

For brevity write $d_u = \deg(u)$ and $d_v = \deg(v)$.

Traversal-based operations. All three are a single graph sweep, so they share the same cost — only the per-vertex neighbour enumeration differs between structures:

	SSSD(u)	CountCC()	DFS(u)
1. Adjacency matrix	$O(n^2)$	$O(n^2)$	$O(n^2)$
2. Adjacency arrays (unsorted)	$O(n + m)$	$O(n + m)$	$O(n + m)$
3. Sorted adjacency arrays	$O(n + m)$	$O(n + m)$	$O(n + m)$
4. Hash-set adjacency lists	$O(n + m)$	$O(n + m)$	$O(n + m)$
5. CSR	$O(n + m)$	$O(n + m)$	$O(n + m)$
6. Adjacency balanced BSTs	$O(n + m)$	$O(n + m)$	$O(n + m)$

CountCommonNeighbours query. This is the operation that distinguishes the structures, since it depends on each one's membership-test mechanism:

	CountCommonNeighbours (u, v)
1. Adjacency matrix	$O(n)$
2. Adjacency arrays (unsorted)	$O(d_u d_v)$
3. Sorted adjacency arrays	$O(d_u + d_v)$
4. Hash-set adjacency lists	$O(\min(d_u, d_v))$
5. CSR	$O(d_u d_v)$
6. Adjacency balanced BSTs	$O(\min(d_u, d_v) \log \max(d_u, d_v))$

Assumptions and justification:

- **Matrix CommonNeighbours** = $O(n)$: scan $w = 1 \dots n$, test $M[u][w] \wedge M[v][w]$.
- **Unsorted arrays / CSR**: for each of u 's neighbours, a membership test in v 's list is a linear scan $O(d_v) \Rightarrow O(d_u d_v)$.
- **Sorted arrays**: merge the two sorted neighbour lists $\Rightarrow O(d_u + d_v)$.
- **Hash sets (traversals)**: worst-case $O(n + m)$ — BFS/DFS only *iterate* each neighbour set ($O(\deg(v))$) and test the vertex-indexed **visited** array ($O(1)$); no hashing lookups are involved.
- **Hash sets (CommonNeighbours)**: iterate the smaller set and do $O(1)$ -expected lookups in the larger $\Rightarrow O(\min(d_u, d_v))$ *expected* (the only operation that relies on hashing).
- **Balanced BST (CommonNeighbours)**: iterate the smaller neighbour set and search each element in the larger tree ($O(\log)$ per search) $\Rightarrow O(\min(d_u, d_v) \log \max(d_u, d_v))$.

Q4. Forced order in all possible DFS traversals

(a) Pseudocode

Characterization. u is visited before v in *every* DFS from s **iff every path from s to v passes through u** (i.e. u *dominates* v).

- *Sufficient*: if every s - v path uses u , the DFS-tree path from s to v also uses u , so u (an ancestor of v) is discovered first.
- *Necessary*: if some s - v path P avoids u , order each vertex's neighbours so DFS walks straight down P ; then v is discovered while u is still undiscovered.

Equivalent test. Remove u , then check whether v is still reachable from s in the graph after deleting vertex u . The search below uses a queue only for this reachability check; any graph traversal (BFS or DFS) works equally. This is a reachability test, not a DFS simulation.

```

MustVisitBeforeDFS(G, s, u, v):
    if v == s: return false    # s discovered first (time 0)
    if u == s: return true     # s first, so discovered before v

    visited <- array[1..n] of false
    visited[u] <- true         # "block" u: traversal never enters it

    Q <- empty queue
    visited[s] <- true
    Q.enqueue(s)
    while Q not empty:
        x <- Q.dequeue()
        for each neighbour y of x:
            if not visited[y]:
                visited[y] <- true
                Q.enqueue(y)

    # v reachable from s WITHOUT u? No => u dominates v.
    if visited[v] == false:
        return true
    else:
        return false

```

(b) Time complexity

One BFS/DFS over the graph plus an $O(n)$ array initialisation: $O(n + m)$.

(c) Worked queries (source $s = A$)

Adjacency:

A: B,C B: A,D,E C: A,G,I D: B,F E: B,F
 F: D,E G: C,H H: G,I I: H,C

Query	u	v	Result	Why? (v reachable from A without u)
1. (A, C, H)	C	H	true	No — $\{G, H, I\}$ reach the rest only via C
2. (A, D, E)	D	E	false	Yes — $A-B-E$ avoids D
3. (A, B, F)	B	F	true	No — $\{D, E, F\}$ reach the rest only via B
4. (A, H, G)	H	G	false	Yes — $A-C-G$ avoids H

Answer. Query results: **true, false, true, false.**

Q5. Longest path in a weighted DAG

(a) Pseudocode

Topologically sort, then DP in **reverse** topological order, where $\text{dist}[u]$ is the length of the longest path *starting* at u , with $\text{next}[u]$ for reconstruction. After the DP pass, reconstruct one optimal path by following **next** pointers.

```

LongestPathDAG(G):
  order <- TopologicalSort(G)  # vertices in topological order
  for each v in V:
    dist[v] <- 0  # one-vertex path has length 0
    next[v] <- NULL

  for i from |order|-1 downto 0:  # reverse topological order
    u <- order[i]
    for each (v, w) in Adj[u]:
      if w + dist[v] > dist[u]:
        dist[u] <- w + dist[v]
        next[u] <- v

  best <- the vertex maximizing dist[.]
  length <- dist[best]

  path <- empty list  # reconstruct longest path
  x <- best
  while x != NULL:
    path.append(x)
    x <- next[x]
  return (length, path)

TopologicalSort(G):  # Kahn's algorithm
  compute indeg[v] for all v
  Q <- all v with indeg[v] = 0
  order <- empty list
  while Q not empty:
    u <- Q.remove()
    order.append(u)
    for each (v, w) in Adj[u]:
      indeg[v] <- indeg[v] - 1
      if indeg[v] = 0: Q.add(v)
  return order

```

(b) Time complexity

Topological sort $O(n + m)$, the DP relaxes each edge once $O(n + m)$, reconstruction $O(n)$. Total: $O(n + m)$.

(c) Worked example

Topological order A, B, C, D, E, F, G, H ; processing in reverse:

dist[H] = 0		
dist[G] = 2	(G→H, 2)	next[G]=H
dist[F] = 3	(F→H, 3)	next[F]=H
dist[E] = 7	(E→H, 7 > E→G:1+2)	next[E]=H
dist[D] = 4	(D→G, 2: 2+2)	next[D]=G
dist[C] = 12	(C→E, 5: 5+7 > C→F:4+3)	next[C]=E
dist[B] = 13	(B→E, 6: 6+7 > B→D:4+4)	next[B]=E
dist[A] = 16	(A→B, 3: 3+13 > A→C:2+12)	next[A]=B

The maximum is $\text{dist}[A] = 16$. Reconstruct: $A \rightarrow B \rightarrow E \rightarrow H$ ($3 + 6 + 7 = 16$).

Answer. Longest path length = 16, achieved by $A \rightarrow B \rightarrow E \rightarrow H$ (edge weights $3 + 6 + 7$).